

**Государственное бюджетное общеобразовательное учреждение города
Москвы "Школа № 1532"**

MUSNET & MYNOTESOPEN

11 класс, ГБОУ Школа №1532,
Ермаков Юрий Дмитриевич,
Жильцов Иван Дмитриевич.
Руководитель: учитель информатики, ГБОУ Школа №1532,
Сергиенко Антон Борисович,
Гладникова Ксения Владимировна.

Москва, 2026

Оглавление

Введение.....	3
Цель и задачи работы.....	4
Методика выполнения работы.....	5
Результат работы и выводы.....	16
Список используемой литературы.....	17

Введение

Музыка — это универсальный язык, который находит отклик в сердцах людей по всему миру. С помощью музыкальных звуков мы можем передавать эмоции, рассказывать истории и создавать атмосферу. Однако, чтобы музыка могла быть точно записана, интерпретирована и воспроизведена, необходимо использовать определенные системы записи. Таким универсальным «алфавитом» музыки стала система нотного письма.

Нотная запись позволяет композиторам и исполнителям передавать свои идеи в визуальной форме. Это своего рода код, который, будучи расшифрованным, превращается в мелодию, гармонию и ритм. Основными символами, используемыми в этой системе, являются ноты, которые представляют собой звуковые высоты и длительности. Поскольку каждая нота имеет свое уникальное положение, это создает возможность для точной передачи музыкального замысла.

Ноты располагаются на специальной графической основе, называемой нотным станом. Нотный стан состоит из пяти параллельных линий и четырех промежутков между ними. Положение символа ноты на нотном стане определяет высоту обозначаемого звука: чем выше нота расположена, тем выше ее звук. Каждая линия и пробел между ними соответствуют определенным музыкальным звукам, что делает нотоносец ключевым инструментом для понимания музыкального произведения.

Однако ноты — лишь каркас произведения, нотный стан может быть дополнен различными знаками и символами, которые указывают на особенности исполнения. Например, динамика, темп и артикуляция — все это можно записать с помощью дополнительных обозначений. Это позволяет зафиксировать не просто последовательность звуков, а их эмоциональную интонацию.

Таким образом, умение читать ноты — это ключ к расшифровке творческого замысла. Оно стирает границы времени и пространства, позволяя современному музыканту воссоздать музыку прошлых веков, а произведениям одного народа обогащать культуру другого. Нотная запись является не просто техническим инструментом, а фундаментальным условием существования и развития мирового музыкального наследия. Именно поэтому алгоритм автоматической обработки нот сложен, актуален и востребован на данный момент. Данный проект мы создаём второй год, но в этом году мы улучшили всё (алгоритм обрабатывает неподготовленное изображение, качество определения увеличилось, и мы улучшили интерфейс) и переписали его с нуля.

Цель и задачи работы

Цель работы: разработать удобную экосистему приложений с библиотекой нот и с системой инструментов для работы с нотами.

Задачи работы:

1. Разработать алгоритм распознавания нот.
2. Разработать алгоритм отрисовки нот.
3. Разработать приложение десктопное приложение.
4. Протестировать десктопное приложение.
5. Перенести и усовершенствовать функции десктопного приложения в Web приложение.

Методика выполнения работы

Этап 1: Исследование и планирование (октябрь 2025 — декабрь 2025):

1.Исследование задачи и требований: Изучение основы музыкальной нотации и существующие приложения. Список необходимых функций, таких как распознавание нот и воспроизведение звуков.

Этап 2: Разработка архитектуры и модулей (декабрь 2025 — январь 2026):

1.Выбор технологий и архитектуры: в качестве платформы по обработке запросов мы выбрали веб-фреймворк Flask, так же для обработки входящего изображения мы использовали библиотеку OpenCV, для перевода нот в midi библиотеку Midiutil, а для обработки midi в mp3 библиотеки FluidSynth и Pydub, для создания десктопной версии приложения на ПК использовали PyQt.

2.Разработка модуля распознавания: Соберите данные, обработайте изображения и реализуйте алгоритм распознавания, проведя тестирование на точность.

Этап 3: Воспроизведение и интерфейс (январь 2026 — февраль 2026):

1.Разработка модуля воспроизведения: Распознанные ноты в MIDI-формат и используйте библиотеки для воспроизведения звуков.

2.Создание интерфейса пользователя: Проектировка и реализация удобного интерфейса, интегрируя его с модулями.

Этап 4: Тестирование и публикация (март 2026 — апрель 2026):

1.Тестирование и доработка: Тестирование приложения.

2.Подготовка к публикации: Документация.

Разбор работы веб-приложения (musnet) и десктоп к приложения (MyNotesOpen)

Веб приложение состоит из нескольких файлов: static, со статичными объектами иконок и др., runserver.py, код для запуска и настройки сервера, __init__.py, код логики страниц, ScanLib.py, для распознавания объектов нот.

Далее я привёл примеры кода `__init__.py` и отрывок `ScanLib.py`(т. к. `ScanLib.py` класс с разными структурами для распознавания)

Листинг 1 — Код инициализации программы `__init__.py`:

```
app = Flask(__name__)

UPLOAD_FOLDER = r'C:\Users\Mi\PycharmProjects\MOSH\
FlaskWebProject1\static\pic'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

@app.route('/', methods=['GET', 'POST'])
def index():
    conn = cuncon()[1]
    if request.method == "POST":
        file = request.files['file']
        print(file)
        if file:
            print("yes")
            filename = secure_filename(file.filename)
            file.save(os.path.join(app.config['UPLOAD_FOLDER'],
filename))
            print(filename)
            int1 =
NoteTrans(f'C:/Users/Mi/PycharmProjects/MOSH/FlaskWebProject1/stat
ic/pic/{filename}')
            MidiNote(int1,
"C:/Users/Mi/PycharmProjects/MOSH/FlaskWebProject1/static/Midi/CC.
mid")
            midi_file_path = r"C:\Users\Mi\PycharmProjects\MOSH\
FlaskWebProject1\static\Midi\CC.mid"
            MidiMp3(midi_file_path)
            os.remove(r"C:\Users\Mi\PycharmProjects\MOSH\
FlaskWebProject1\static\Wav\mpshka.wav")
            return redirect("home", 301)
        else:
            return render_template('layout.html')
    return render_template('layout.html')
@app.route('/home')
def homeindex():
    return render_template('about.html')
```

Всего есть две страницы `Localhost/` - для всей обработки и `Localhost/home` для отображения аудио файла. `Localhost/` - использует в себе ещё три функции: `NoteTrans` — для определения нот, `MidiNote` – для перевода нот в `mid` и `MidiMp3` — для сохранения и создания `mp3` файла.

Листинг 2 — Код обработки изображения ScanLib.py:

```
def scan_note(file_path, algorithm, size=5, binar=100, blur_s=3,
axis="xys"):
    if file_path == "":
        return []
    img = cv2.imread(file_path, cv2.IMREAD_GRAYSCALE)
    blur = cv2.GaussianBlur(img, (blur_s + 2, blur_s + 2), 0)
    edges = cv2.Canny(blur, 50, 150) # 50 и 150 — пороги
чувствительности
    contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    true_contour = np.array([[0, 0], [0, 0], [0, 0], [0, 0]])
    for cnt in contours:
        if cv2.contourArea(cnt) > 100 and cv2.contourArea(cnt) >
cv2.contourArea(true_contour):
            true_contour = cnt

    x, y, w, h = cv2.boundingRect(true_contour) # Получаем
координаты прямоугольника
    if algorithm == "clipping":
        clear_img = img[y:y + h, x:x + w]
    else:
        clear_img = img
    blur = cv2.GaussianBlur(clear_img, (blur_s, blur_s), 0)
    edges = cv2.Canny(blur, 50, 150)
    contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    contours_true = []
    for cnt in contours:
        if cv2.contourArea(cnt) > 100:
            contours_true.append(cnt)

    merged_true = merged_contours(contours_true)
    for i in contours_true:
        x, y, w, h = cv2.boundingRect(i)

    notes = []
    imges, images_edges, _ = distribute_cut(merged_true,
clear_img)
    for cny in range(len(images_edges)):
        lst_notelines = []
        lst_note = []

        a = (len(imges[cny][1]) * 0.8)
        b = (len(imges[cny]) * 0.1)
        lines = cv2.HoughLinesP(images_edges[cny], 1, np.pi / 180,
threshold=20, minLineLength=a, maxLineGap=50)
```

```

merged_lines = merge_lines(lines, 1, 1)
merged_lines = merged_noty_line(merged_lines, b, "up")
for i in range(len(merged_lines) - 1):
    x1, y1, x2, y2 = merged_lines[i][0]
    x3, y3, x4, y4 = merged_lines[i + 1][0]
    if (abs(y3 - y1) + abs(y4 - y2)) / 2 <= b and
len(merged_lines) != 5:
        merged_lines = merged_noty_line(merged_lines, b,
"down")
        break
attempts = 0
while len(merged_lines) != 5 and attempts <= size:
    if len(merged_lines) > 5:
        n = [] # числа разности
        for i in range(len(merged_lines) - 1):
            x1, y1, x2, y2 = merged_lines[i][0]
            x3, y3, x4, y4 = merged_lines[i + 1][0]
            n.append((abs(y3 - y1) + abs(y4 - y2)) / 2)
        b = max(n) * 1.3
        merged_lines = merged_noty_line(merged_lines, b)
    elif 1 < len(merged_lines) < 5:
        n = [] # числа разности
        for i in range(len(merged_lines) - 1):
            x1, y1, x2, y2 = merged_lines[i][0]
            x3, y3, x4, y4 = merged_lines[i + 1][0]
            n.append((abs(y3 - y1) + abs(y4 - y2)) / 2)
        b = min(n) * 1.1
        merged_lines = completion_noty_line(merged_lines,
b)

        attempts += 1

for line in merged_lines:
    x1, y1, x2, y2 = line[0]
    cv2.line(imges[cny], (x1, y1), (x2, y2), (255, 255,
255), 2)

    lst_notelines.append(y1)
lst_notelines.sort()

_, thresh = cv2.threshold(imges[cny], binar, 255,
cv2.THRESH_BINARY_INV)
images_edges_ = thresh

contours, _ = cv2.findContours(images_edges_,
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
true_contour = []

for cnt in contours:
    if cv2.contourArea(cnt) > 1: # Игнорируем мелкие

```

объекты

```
x, y, w, h = cv2.boundingRect(cnt)
perimeter = cv2.arcLength(cnt, True)
approx = cv2.approxPolyDP(cnt, 0.02 * perimeter,
True)

    if len(approx) > 6:
        (x, y), radius = cv2.minEnclosingCircle(cnt)
        area = cv2.contourArea(cnt)
        circularity = 4 * np.pi * area / (perimeter **
2)

        # Круглость близка к 1? (1 — идеальная
окружность)

        if 0.1 < circularity < 1.5:
            print("нота")
            true_contour.append(cnt)
contours_note = merged_noty_contours(true_contour, axis,
20, 20, 10)
lst_note = contours_note
for i in contours_note:
    x, y, w, h = cv2.boundingRect(i)
    cv2.rectangle(imges[cny], (x, y), (x + w, y + h), (0,
255, 0), 2)

    x, y, w, h = map(int, (x, y, w, h))
    cl = imges[cny][0:, x - (2 * w):x + (3 * w)]
    ing = images_edges_[0:, x - (2 * w):x + (3 * w)]
    lines = cv2.HoughLinesP(ing, 1, np.pi / 2,
threshold=20, minLineLength=3, maxLineGap=50)
    merged_lines = merge_lines(lines)
    for line in merged_lines:
        x1, y1, x2, y2 = line[0]
        cv2.line(cl, (x1, y1), (x2, y2), (0, 255, 0), 2)

# распределение нот по высотам

if len(lst_notelines) == 0:
    continue
n = []
for i in range(len(lst_notelines) - 1):
    n.append(abs(lst_notelines[i] - lst_notelines[i + 1]))
if len(n) > 0:
    difference_not = (min(n) * 0.9) / 2
else:
    difference_not = (20 * 0.9) / 2
note_res = []
for note in lst_note:
    x, y, w, h = cv2.boundingRect(note)
    point_note = y + (h / 2)
    note_type = "None"
```

```

        if lst_notelines[0] - difference_not > point_note:
            note_type = "c"
        elif lst_notelines[0] - difference_not < point_note <
lst_notelines[0]:
            note_type = "d"
        elif lst_notelines[0] - difference_not < point_note <
lst_notelines[0] + difference_not:
            note_type = "e"
        elif lst_notelines[0] < point_note < lst_notelines[1]:
            note_type = "f"
        elif lst_notelines[1] - difference_not < point_note <
lst_notelines[1] + difference_not:
            note_type = "g"
        elif lst_notelines[1] < point_note < lst_notelines[2]:
            note_type = "a"
        elif lst_notelines[2] - difference_not < point_note <
lst_notelines[2] + difference_not:
            note_type = "h"
        old_x, old_y, old_w, old_h =
cv2.boundingRect(merged_true[cny])
        contour_note = [old_x + x, old_y + y, w, h]
        if note_type != "None":
            note_res.append([note_type, contour_note])
        note_res.sort(key=lambda x: x[1][0])
        notes.append(note_res)
    if len(notes[0]) >= 1:
        notes.sort(key=lambda x: x[0][1][1])
    return notes

```

Вот обобщённое объяснение логики функции:

1. Бинаризация изображения делаем его черно белым
2. Выявляем горизонтальные и вертикальные линии
3. Соединяем линии, эрозия и дилатация для улучшения качества
4. Находим контуры и фильтруем по размеру. Чтоб отсеять объекты, которые больше или меньше нот.
5. Сортируем по координатам слева направо полученные контуры чтобы понять порядок нот
6. Сортируем по координатам слева направо полученные контуры чтобы понять порядок нот
7. определяются типы нот. Рассматривается высота и именно по ней определяется нота. Добавляем ноты в список.

Рассмотрим каждый пункт подробнее:

В начале (в пункте 1) мы бинаризуем изображение, то есть превращаем картинку в чёрно-белый формат с чёткими контурами (Рисунок 1). Затем определяем контуры с помощью OpenCV и в зависимости от алгоритма обрезаем лишнее или анализируем всё.

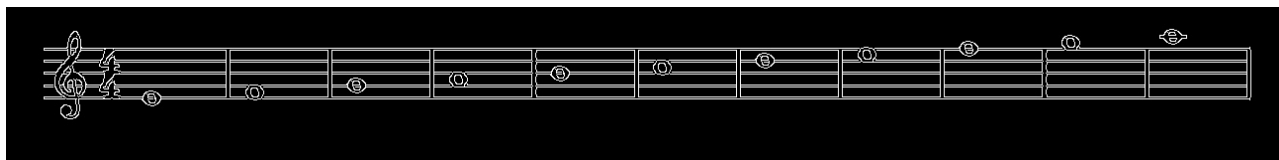


Рисунок 1 — пример бинаризации.

Дальше (в пункте 2) опять ищем контуры только теперь обрабатываем их все. Для этого в каждом контуре мы проводим заново бинаризацию, ищем длинные линии нотного стана и, если хоть какие-то линии были найдены, проверяем их по специальным условиям и достраиваем дополнительные линии (для ускорения процесса, а также из-за того, что они не всегда определяются, потому что они бывают узкие). Определив линии записываем их координаты записываем в массив, сортируем и убираем эти линии с изображения т. к. они мешают определению нот. И получаем такое изображение (Рисунок 2).



Рисунок 2 — пример обработанных строк нотного листа:

А теперь (в пункте 3) подготовим изображение к обработке. Для этого размываем картинку (чтобы убрать шум и другие дефекты фотографии), а после улучшаем изображение с помощью алгоритмов эрозии и дилатации и заново проведем бинаризацию. Теперь, когда мы подготовили изображения, мы можем начать поиск нот.

Найдём (в пункте 4–6) заново контуры и теперь отберём их. Мы проверим их размер, круглые они или нет и соединим контуры, которые находятся близко или совпадают (Рисунок 3). Так мы получим массив нот с их координатами.



Рисунок 3 — пример визуализации найденных объектов нот.

Наконец (в пункте 7) распределим ноты по высотам с помощью линий нотного стана, если они слишком высокие или низкие мы просчитаем их высоту, и запишем ноты в буквенном обозначении для последующей обработки.

Листинг 3 — Код обработки массива нот ScanLib.py:

```
def MidiNote(notes, location):
    SlovNote = {'c': 60, 'd': 62, 'e': 64, 'f': 65, 'g': 67, 'a':
69, 'h': 71, 'c1': 72, 'd1': 74, 'e1': 76, 'f1': 77,
                'g1': 79, 'a1': 81, 'h1': 83}
    finally = []
    for i in notes:
        finally.append(SlovNote[i])
    print(finally)
    degrees = finally # MIDI note number
    track = 0
    channel = 0
    time = 0 # In beats
    duration = 1 # In beats
    tempo = 60 # In BPM
    volume = 100 # 0-127, as per the MIDI standard

    MyMIDI = MIDIFile(1) # One track, defaults to format 1 (tempo
track
# automatically created)
    MyMIDI.addTempo(track, time, tempo)

    for pitch in degrees:
        MyMIDI.addNote(track, channel, pitch, time, duration,
volume)
        time = time + 1
    with open(location, "wb") as output_file:
        MyMIDI.writeFile(output_file)
```

В этой функции мы с помощью списка и словаря определяем midi высоты. После создавая на их основе midi файл.

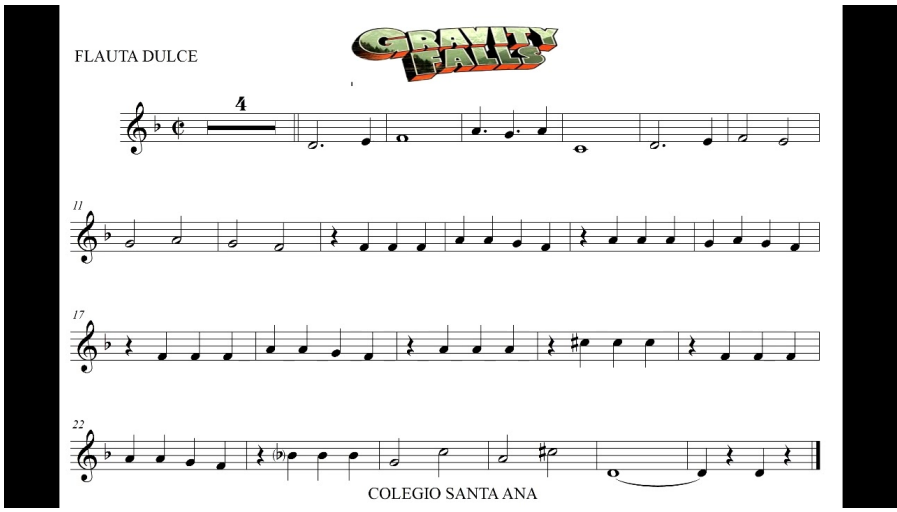
Листинг 4 — Код создания Wav и Mp3 файла ScanLib.py:

```
def MidiMp3(midi_file_path,
            soundfont_path=r'l_Giga_piano.sf2',
            mp3_file_path=r'',
            wav_path=r''):
    fs = FluidSynth(sound_font=soundfont_path, sample_rate=96000)
    fs.midi_to_audio(midi_file_path, wav_path)
    audio = AudioSegment.from_wav(wav_path)
    audio.export(mp3_file_path, format='mp3')

def MidiWav(midi_file_path,
            soundfont_path=r'l_Giga_piano.sf2',
            wav_file_path=r''):
    fs = FluidSynth(sound_font=soundfont_path, sample_rate=96000)
    fs.midi_to_audio(midi_file_path, wav_file_path)
    audio = AudioSegment.from_wav(wav_file_path)
    audio.export(wav_file_path, format='wav')
```

В данной последней функции мы с помощью midi файла SoundFound библиотеке звуков и библиотек конвертируем midi в аудио файл.

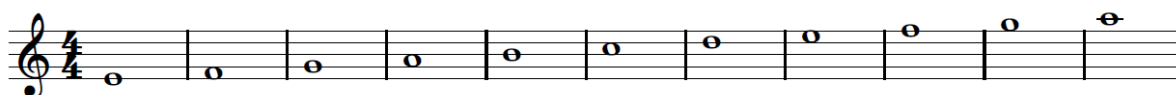
Результат работы всего алгоритма вы можете прослушать и просмотреть по ссылке: <https://disk.yandex.ru/d/PO98ilomsdj2oA>

Работа приложения			
Загруженное фото			
Полученные файлы	Файл mp3	Файл Wav	Файл MIDI
			

Десктоп приложение состоит из нескольких файлов: SetupApp.py, PyQTapp.py, ScanLib.py, TransNote.py. Данное созданное мной приложение Open Source и выложено мной на сайт GitHub. Там вы можете ознакомиться подробнее с кодом проекта и работой приложения.

Ссылка:

<https://github.com/Web-your/-QT-musical-note-recognition-application/tree/main/MusicNotesQT>



FLAUTA DULCE



COLEGIO SANTA ANA

Undertale: Finale

Toby Fox

$\text{♩} = 180$

The musical score is written for a single melodic line in treble clef, key of D major (two sharps), and 4/4 time. The tempo is marked as quarter note = 180. The score consists of 33 measures, divided into eight staves. Measures 1-12 are marked *mp* (measures 1-4) and *f* (measures 5-12). Measures 13-20 are marked *mf* (measures 13-14) and *p* (measures 15-20). Measures 21-24 are marked *ff*. Measures 25-28 are marked *ff*. Measures 29-32 are marked *ff*. Measure 33 is marked *ff*. The melody is characterized by a repeating eighth-note pattern in the first half of each measure, with a descending eighth-note pair in the second half. The key signature has two sharps (F# and C#). The score ends with a double bar line and a repeat sign.

mp

5

9

f

13

mf

p

21

ff

25

ff

29

ff

33

ff

Рисунок 4 — Фото, использованные для тестирования.

Результат работы и выводы

Результатом работы станет web-приложение с хранилищем нот и функциями для распознавания нотного листа и его воспроизведения, которое позволит пользователям автоматически распознавать ноты с изображений, визуализировать их и воспроизводить музыку в формате MIDI. Приложение будет иметь интуитивно понятный интерфейс, поддерживающий загрузку изображений и управление воспроизведением. Также предусмотрены функции редактирования нот и настройки темпа. Документация поможет пользователям разобраться с функционалом, а поддержка обеспечит решение возникающих вопросов.

Список используемой литературы

1. Бреан Аре, Скейе Гейр Ульве Музыка и мозг: как музыка влияет на эмоции, здоровье и интеллект Бреан Аре, Скейе Гейр Ульве – Москва: Альпина Паблишер, 2020. – 275.
2. В. Вахромеев Элементарная теория музыки: В. Вахромеев – Москва: Государственное музыкальное издательство, 1961. – 255.
3. Мигель Гринберг Разработка веб-приложений с использованием Flask на языке Python Мигель Гринберг – Москва: ДМК Пресс, 2016. – 274.
4. Майк МакГрат JavaScript для начинающих Майк МакГрат – Москва: БОМБОРА, 2023. – 232.
5. Владимир Дронов, Николай Прохоренок Python 3 и PyQt 5. Разработка приложений Владимир Дронов, Николай Прохоренок – Москва: БХВ-Петербург, 2016. – 830.